Technical University of Crete
School of Electrical and Computer Engineering
Course: Optimization
Instructor: Athanasios P. Liavas
Exercise 2 (100/1000)
Report Delivery Date: November 18, 2025

In the first part of the exercise, we will solve a convex quadratic optimization problems.

A. We start with a simple theoretical problem.

(a) (10) Let $f : \mathbb{R}^n \to \mathbb{R}$. For fixed $\mathbf{x} \in \mathbb{R}^n$ and $m > 0$, let $g_{\mathbf{x}} : \mathbb{R}^n \to \mathbb{R}$ be defined as

$$g_{\mathbf{x}}(\mathbf{y}) := f(\mathbf{x}) + \nabla f(\mathbf{x})^T(\mathbf{y} - \mathbf{x}) + \frac{m}{2}\|\mathbf{y} - \mathbf{x}\|_2^2. \tag{1}$$

i. (5) Compute $\nabla g_{\mathbf{x}}(\mathbf{y})$.

ii. (5) Compute the optimal point, $\mathbf{y}_* = \underset{\mathbf{y}}{\operatorname{argmin}}\, g_{\mathbf{x}}(\mathbf{y})$, and the optimal value, $g_{\mathbf{x}}(\mathbf{y}_*)$.

B. We proceed to the solution of convex quadratic problems. Our goal is to study the behavior of the gradient method, with exact and backtracking line search.

(a) (40) Consider the problem

$$\underset{\mathbf{x}\in\mathbb{R}^n}{\text{minimize}}\ f(\mathbf{x}) = \tfrac{1}{2}\,\mathbf{x}^T\mathbf{P}\mathbf{x} + \mathbf{q}^T\mathbf{x}, \tag{2}$$

where $\mathbf{P} \in \mathbb{R}^{n\times n}$, $\mathbf{P} = \mathbf{P}^T \succ \mathbf{O}$ and $\mathbf{q} \in \mathbb{R}^n$ (indicative values of n are $n = 2, 50, 10^2, 10^3$.)

i. A random positive definite matrix $\mathbf{P}$ can be constructed in many ways. To fully control the condition number of the matrix, we can act as follows. Every positive definite $\mathbf{P}$ can be expressed as

$$\mathbf{P} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T \tag{3}$$

with $\mathbf{U}, \mathbf{\Lambda} \in \mathbb{R}^{n\times n}$, $\mathbf{U}\mathbf{U}^T = \mathbf{U}^T\mathbf{U} = \mathbf{I}_n$,[1] and $\mathbf{\Lambda} = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$, with $\lambda_i > 0$, for $i = 1, \ldots, n$. The columns of $\mathbf{U}$ are the eigenvectors of $\mathbf{P}$ and the elements of the diagonal of $\mathbf{\Lambda}$ are the eigenvalues of $\mathbf{P}$.

[1] Matrices that satisfy this property are called **orthogonal**.

(a) To create a random orthonormal $\mathbf{U}$, create a random $(n \times n)$ matrix $\mathbf{A}$ as follows

$$\mathtt{A} = \mathtt{randn}(\mathtt{n}, \mathtt{n});$$

and then compute its singular value decomposition as

$$[\mathtt{U}, \mathtt{S}, \mathtt{V}] = \mathtt{svd}(\mathtt{A});$$

Compute $\mathtt{U} * \mathtt{U}'$ and $\mathtt{U}' * \mathtt{U}$. What do you observe?

(b) To construct the eigenvalues λ_i, for $i = 1, \ldots, n$, select the smallest and largest, $\lambda_{\min}$ and $\lambda_{\max}$, respectively. You can create the other eigenvalues in any way you wish. For example, you can create $(n - 2)$ random numbers, uniformly distributed in the interval $[\lambda_{\min}, \lambda_{\max}]$ using the command

$$\mathtt{z} = \lambda_{\min} + (\lambda_{\max} - \lambda_{\min}) * \mathtt{rand}(\mathtt{n} - 2, 1); \tag{4}$$

and the vector of the n eigenvalues as

$$\mathtt{eig_P} = [\lambda_{\min}; \lambda_{\max}; \mathtt{z}]; \tag{5}$$

Matrix $\mathbf{\Lambda}$ may be constructed as

$$\mathbf{\Lambda} = \mathtt{diag}(\mathtt{eig_P}); \tag{6}$$

The condition number of the problem is $\mathcal{K} := \frac{\lambda_{\max}}{\lambda_{\min}}$.

ii. (5) Construct random vector $\mathbf{q}$ and random positive definite matrix $\mathbf{P}$, with condition number $\mathcal{K}$ (indicative values, $\mathcal{K} = 10, 10^2, 10^3$).

iii. (5) Solve problem (2) using the closed-form solution, and compute the optimal point, $\mathbf{x}_*$, and the optimal value, $p_* = f(\mathbf{x}_*)$.

iv. Solve problem (2) using the `fmincon` function of matlab.

v. (15) Solve problem (2) using the gradient algorithm (with exact and back-tracking line search). How many iterations are sufficient for convergence, if $\mathcal{K} = 1$ and you use the exact line search?

vi. (5) For $n = 2$, construct a contour plot, with levels the values $f(\mathbf{x}_k)$. Then, on top of this plot, plot the trajectories of $\{\mathbf{x}_k\}$ produced by the two algorithms. (For problems with large condition number, you must observe the "zig-zag" effect.)

vii. (5) Plot quantity $\log(f(\mathbf{x}_k) - p_*)$, produced by the two algorithms, versus k. What is the slope of this plot? What do you observe for different condition numbers?

viii. (5) Using the convergence analysis results for strongly convex functions, and the values of $f(\mathbf{x}_0)$, p_* and ϵ, compute the minimum number of iterations that guarantees solution within accuracy ϵ, i.e., $f(\mathbf{x}_k) - p_* \leq \epsilon$, and compare it with the number of iterations performed by the algorithms. What do you observe?

In the second part of this exercise, we will study the problem of (linear) data classification via Logistic Regression (LR). For the notation, please, check my Lecture Notes (50).

A. **Data Generation:** We generate the (approximately) linearly separable dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, where $\mathbf{x}_i \in \mathbb{R}^N$ and $y_i \in \{1, 0\}$. The data generation I propose proceeds as follows:

1. Generate random $\mathbf{w} \in \mathbb{R}^N$, and $b \in \mathbb{R}$.
2. (5) Compute a point, say $\mathbf{x}_0$, of the hyperplane $\mathbb{H}_{b,\mathbf{w}}$.
3. For $i = 1, \ldots, n$, generate random class $y_i \equiv \mathtt{y(i)} \in \{0, 1\}$.
4. For $i = 1, \ldots, n$, generate the data points $\mathbf{x}_i \equiv \mathtt{X(:, i)}$ as follows:

$$\mathbf{x}_i = \begin{cases} \mathbf{x}_0 + \mathbf{w} + \mathtt{std} * \mathtt{randn(N, 1)}, & \text{if } y_i = 1, \\ \mathbf{x}_0 - \mathbf{w} + \mathtt{std} * \mathtt{randn(N, 1)}, & \text{otherwise,} \end{cases} \tag{7}$$

   where parameter `std` is the standard deviation of the noise and controls the linear separability of the data. "Small" `std` leads, with high probability, to separable data, while the opposite happens for "large" `std`.
5. (10) If $N = 2$, plot the line defined by $\mathbf{w}$ and b, as well as the data points $\mathbf{x}_i$, for $i = 1, \ldots, n$, each class with different color. Check, visually, the separability of the data and perform experiments with different values of $\mathbf{w}$, b and `std`.
6. Define $\boldsymbol{\theta} := (b, \mathbf{w})$ and augment appropriately the data points by inserting -1 at their first position. You may call the new data matrix as `X_augm`.

B. **Classification via Logistic Regression:**

1. (5) Choose $\lambda > 0$ (for example, $\lambda = 10^0, 10^{-1}, 10^{-2}$) and solve the l_2-regularized LR problem via the `fmincon`. Denote the solution as `theta_*`. If $N = 2$, plot the data, the original line used for data generation, and the line computed via the LR. Perform experiments with different values of λ. What do you observe?
2. (20) Solve the l_2-regularized LR problem via Gradient Descent (GD) with Backtracking line search. Use any terminating condition you think is appropriate. In the k-th iteration, compute the "error" `norm(theta_k - theta_*)`.

Solutions

Patrikaki Aristea - 2023030078

Part 1: Quadratic Optimization

A. (a)

$$g_{\mathbf{x}}(\mathbf{y}) := f(\mathbf{x}) + \nabla f(\mathbf{x})^T(\mathbf{y} - \mathbf{x}) + \frac{m}{2}\|\mathbf{y} - \mathbf{x}\|_2^2$$

i. To find the gradient with respect to y, we differentiate each term:

- The term $f(x)$ is a constant with respect to y, so its derivative is 0.
- The term $\nabla f(x)^T(y - x) = \nabla f(x)^T y - \nabla f(x)^T x$. The derivative of $\nabla f(x)^T y$ with respect to y is $\nabla f(x)$.
- The term $\nabla f(x)^T x$ is a constant, so its derivative is 0.
- For the term $\frac{m}{2}||y - x||_2^2$, we use the rule $\nabla_y ||y - x||_2^2 = 2(y - x)$. So, $\nabla_y \left(\frac{m}{2}||y - x||_2^2\right) = \frac{m}{2} \cdot 2(y - x) = m(y - x)$.

Combining these results, the gradient is:

$$\nabla g_x(y) = \nabla f(x) + m(y - x)$$

ii. To find the optimal point y_*, we set the gradient we just found to zero:

$$\nabla g_x(y_*) = 0$$

$$\nabla f(x) + m(y_* - x) = 0$$

Solving for y_*, we get:

$$y_* = x - \frac{1}{m}\nabla f(x)$$

Now, we substitute y_* back into the original function $g_x(y)$ to find the optimal value $g_x(y_*)$:

$$g_x(y_*) = f(x) + \nabla f(x)^T(y_* - x) + \frac{m}{2}||y_* - x||_2^2$$

We use the experssion $y_* - x = -\frac{1}{m}\nabla f(x)$ that we found above:

$$g_x(y_*) = f(x) + \nabla f(x)^T\left(-\frac{1}{m}\nabla f(x)\right) + \frac{m}{2}\left|\left|-\frac{1}{m}\nabla f(x)\right|\right|_2^2$$

$$g_x(y_*) = f(x) - \frac{1}{m}||\nabla f(x)||_2^2 + \frac{m}{2}\left(\frac{1}{m^2}||\nabla f(x)||_2^2\right)$$

$$g_x(y_*) = f(x) - \frac{1}{2m}||\nabla f(x)||_2^2$$

B. i. (a) MATLAB results for random orthonormal U:

```
A =
    0.3190    3.5780    0.7250   -0.1240
   -1.3080    2.7690   -0.0630    1.4900
   -0.4340   -1.3500    0.7150    1.4090
    0.3430    3.0350   -0.2050    1.4170

U =
    0.6030   -0.4130   -0.3630   -0.5780
    0.5370    0.5210    0.6300   -0.2080
   -0.1670    0.7300   -0.5720   -0.3360
    0.5660    0.1610   -0.3790    0.7140

U * U' =
     1     0     0     0
     0     1     0     0
     0     0     1     0
     0     0     0     1

U' * U =
     1     0     0     0
     0     1     0     0
     0     0     1     0
     0     0     0     1
```

Observation: Both UU^T and U^TU are identity matrices, confirming that U is orthonormal.

(b) MATLAB results for eigenvalues and Lambda matrix:

```
lam_(min) = 1
lam_(max) = 10
Condition number K = 10

Eigenvalues (lam_i):
    1.0000
    2.1430
    8.3330
    9.1520
    9.2200
   10.0000

Lambda = diag(lam_i):
```

```
    1.0000         0         0         0         0         0
         0    2.1430         0         0         0         0
         0         0    8.3330         0         0         0
         0         0         0    9.1520         0         0
         0         0         0         0    9.2200         0
         0         0         0         0         0   10.0000
```

Observation: We constructed $\Lambda = \text{diag}(\lambda_1, \ldots, \lambda_n)$ and verified that the condition number of the problem is $K = \frac{\lambda_{\max}}{\lambda_{\min}}$.

ii. MATLAB results for random P matrix and q vector:

```
P matrix (First 5x5 block):
   60.4672   -7.6470    0.7901   -2.9594    3.7157
   -7.6470   46.2021    6.4125   -2.7230   -3.2060
    0.7901    6.4125   52.3963   -5.9958   -4.7492
   -2.9594   -2.7230   -5.9958   61.5076   -2.7627
    3.7157   -3.2060   -4.7492   -2.7627   61.4536

q vector (First 5 elements):
    2.5350
    0.4386
    0.4375
   -0.8377
   -1.3075
```

iii. MATLAB results for closed-form solution:

```
Optimal Value p* (f(x*)): -1.224592095375321

Optimal Point x* (first 5 elements):
   -0.0303
    0.1121
   -0.0505
   -0.0578
    0.0868
```

iv. MATLAB results for fmincon solution:

```
Optimal point x_fmincon: -0.097464    -0.10704    -0.087575     0.21997
0.038325    -0.16282    0.047117    0.086238    0.065965    0.093618
```

```
0.025329    -0.20224    -0.05243    -0.10875    -0.17019   -0.045461
-0.0059433    0.036791     0.44262   -0.023381    0.024843   -0.079468
-0.1257    0.014922   -0.014742    -0.14629    ....
```

```
We observe that the optimal point and value from fmincon are very
close to the closed-form solution.
```

v. 1. Exact Line Search (ELS)For a quadratic function, the exact step size α_k is known in closed form:

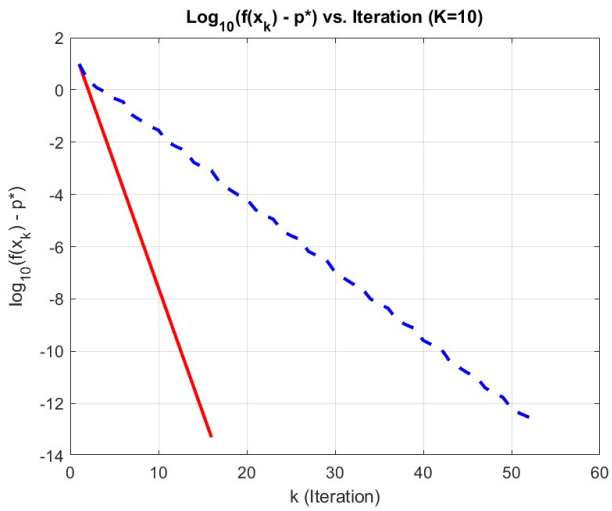
$$\alpha_k = \frac{\nabla f(x_k)^T \nabla f(x_k)}{\nabla f(x_k)^T P \nabla f(x_k)}$$

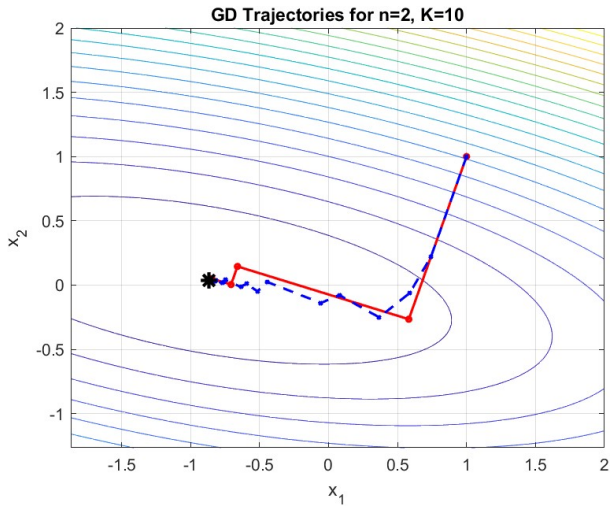
2. Backtracking Line Search (BLS)The step size α_k is chosen by starting with $\alpha = 1$ and repeatedly reducing it by a factor $\beta \in (0, 1)$ until the Armijo condition is satisfied:

$$f(x_k - \alpha \nabla f(x_k)) \leq f(x_k) - \gamma \alpha ||\nabla f(x_k)||_2^2$$

```
MATLAB results (for K = 10^3 with ELS):

Exact Line Search reached max_iter (5000).
Backtracking Line Search converged in 633 iterations.
Final ELS f(x_k): -0.68579244
Final BLS f(x_k): -0.68579244

MATLAB results (for K = 1 with ELS):

Exact Line Search converged in 1 iterations.
Backtracking Line Search converged in 633 iterations.
Final ELS f(x_k): -31.62993406
Final BLS f(x_k): -31.62993406

We observe that for K = 1, ELS converged in just 1 iteration, as
expected for well-conditioned problems.
```

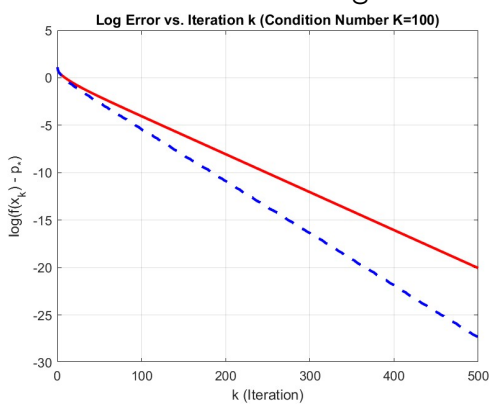
vi. MATLAB results for contour plots (K = 10 and K = 10^3):

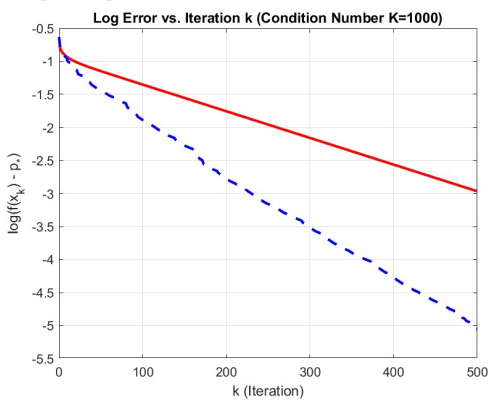




Observation: For large condition numbers $\mathcal{K}$, the level sets of $f(x)$ are elongated ellipses. The gradient descent steps will exhibit a "zig-zag" effect

vii. `MATLAB results for log(f(x_k) - p*) plots (K = 10 and K = 10^3):`





The convergence of the gradient method for strongly convex quadratic functions is linear (Q-linear): $f(x_k) - p_* \leq C \cdot \rho^k$.
Taking the logarithm: $\log(f(x_k) - p_*) \leq \log(C) + k\log(\rho)$.
The plot of $\log(f(x_k) - p_*)$ vs. k is approximately a straight line whose slope is $\log(\rho)$.
The convergence rate ρ is related to the condition number $\mathcal{K}$ by $\rho \leq \frac{\mathcal{K}-1}{\mathcal{K}+1}$.

Small $\mathcal{K}$ (e.g., 10): ρ is small (closer to 0), so $\log(\rho)$ is a large negative number. The slope is steep.
Large $\mathcal{K}$ (e.g., 10^3): ρ is close to 1, so $\log(\rho)$ is close to 0. The slope is gentle.

viii. The minimum number of iterations k that guarantees the solution within accuracy ϵ is given by the convergence analysis for strongly convex functions:

$$f(x_k) - p_* \leq \rho^k (f(x_0) - p_*) \leq \epsilon$$

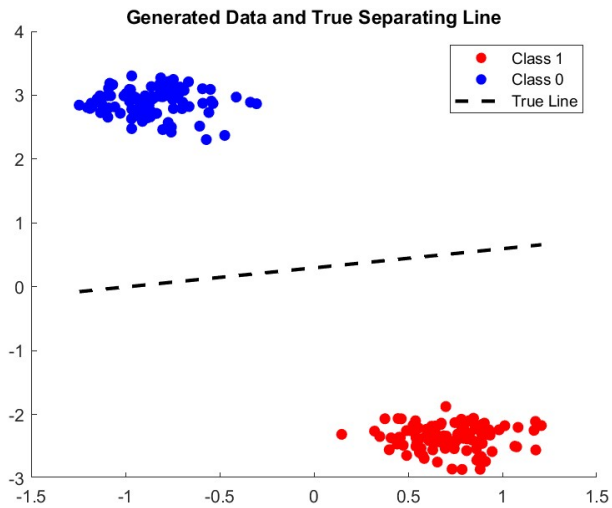
Solving for k, using the worst-case convergence rate $\rho = \frac{\mathcal{K}-1}{\mathcal{K}+1}$:

$$k \geq \frac{\log(\epsilon) - \log(f(x_0) - p_*)}{\log(\rho)} = \frac{\log\left(\frac{\epsilon}{f(x_0)-p_*}\right)}{\log\left(\frac{\mathcal{K}-1}{\mathcal{K}+1}\right)}$$

The theoretical minimum number of iterations k provides an upper bound. The actual number of iterations performed by the algorithms will typically be fewer than the theoretical prediction, especially when the starting point x_0 is already close to x_*.

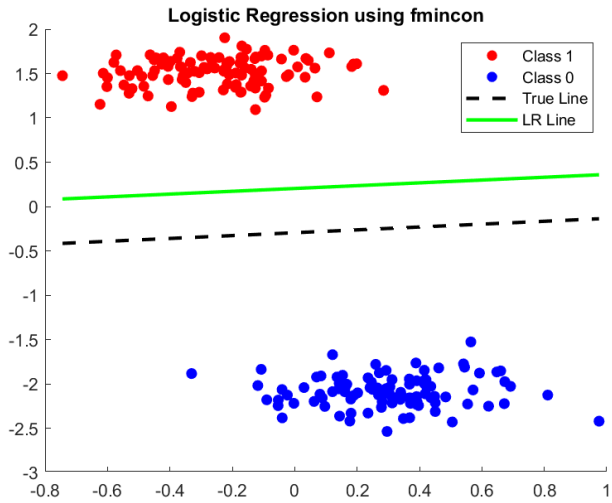
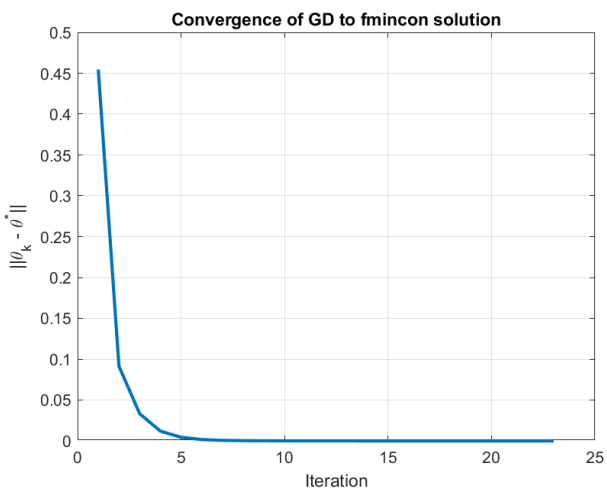
Part 2: Logistic Regression

A. `We generated a synthetic binary dataset by choosing random w and b to define a separating line.`
`A point on the line was computed, and data were sampled around x0 + w and x0 - w depending on the class.`
`Gaussian noise controlled by "std" determines how linearly separable the data are.`
`The plot shows both classes and the true separating line.`



B. 1. `The logistic regression line is different from the true line used`

to generate the data.
Because of the L2-regularization, the LR solution shrinks the parameters and shifts the boundary slightly upward.
Even though it does not match the true line exactly, it still separates the two classes correctly.



2. Using gradient descent with backtracking line search, the parameter vector converges towards the fmincon solution.
This is shown by the decreasing norm ||theta_k - theta_*|| over the iterations, indicating that GD successfully approximates the optimal logistic regression parameters.